

Guardian Eye RAG Pipeline Technical Documentation

Detailed thesis-ready documentation for the three-RAG backend architecture

This document describes the complete Retrieval-Augmented Generation (RAG) layer designed for Guardian Eye, a violence-detection system that receives classifier output, produces grounded explanations, stores queryable incident memory, and generates cautious country-specific legal consequence summaries. It is written as a thesis-ready technical chapter and can be adapted directly into the project methodology, system design, and implementation sections.

1. Executive Summary

Guardian Eye separates visual classification from language generation. The classifier is the authority for the final violence or non-violence verdict and the confidence score. The RAG layer does not re-decide the label. Instead, it explains, stores, retrieves, and contextualizes the classifier decision using grounded evidence. The final architecture contains three RAG stores: Explanation/Reference RAG, Incident Memory/Historical RAG, and Legal Consequences RAG.

- RAG #1: Explanation/Reference RAG grounds the narrative in a small curated reference corpus and evidence packet.
- RAG #2: Incident Memory RAG stores previous incidents in SQLite and a vector index for later historical questions.
- RAG #3: Legal Consequences RAG retrieves country-specific legal sources and generates cautious, non-advisory legal consequence summaries.
- TranslateGemma is used as a wrapper for Arabic/English user-facing translation while keeping the retrieval stores stable.
- Guardrails prevent hallucinations, verdict contradiction, unsupported exact action claims, guilt declarations, guaranteed punishment, and legal advice.

Guardian Eye Complete RAG Pipeline

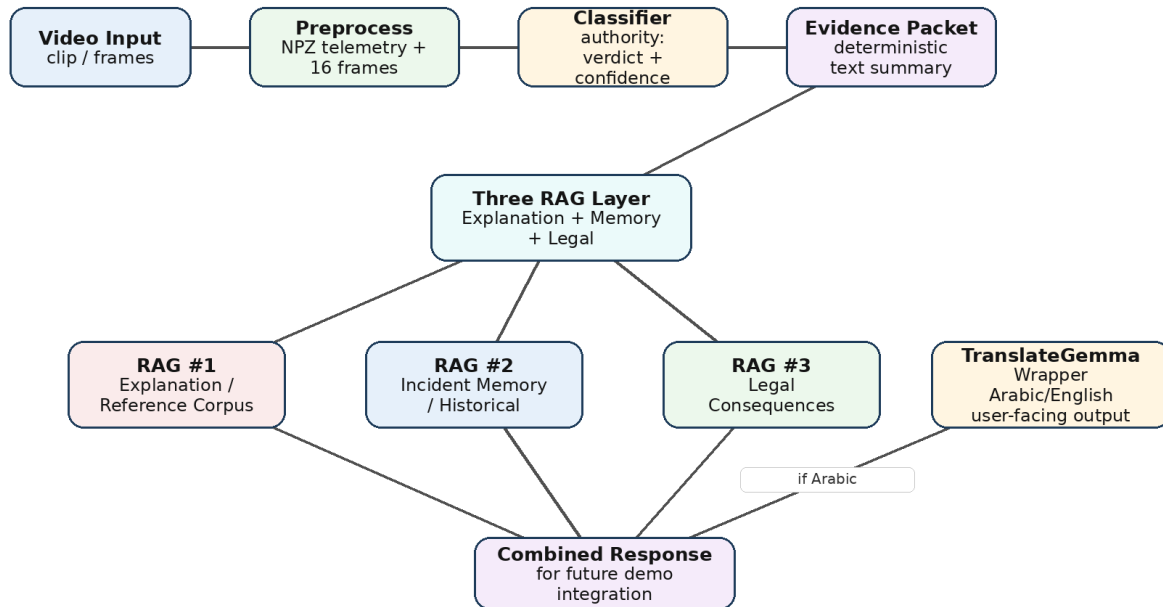


Figure 1. Full Guardian Eye RAG pipeline.

2. Design Principles

Principle	Implementation meaning
Classifier authority	The classifier output is the final source of truth for verdict and confidence. LLM/VLM components are narrators only.
Grounded generation	Generated responses must be based on evidence packets, retrieved references, stored incident records, or retrieved legal chunks.
Local/offline-friendly	The project uses local Python modules, FAISS, SQLite, and load-on-demand models. Tests use mocks to avoid downloads.
Integration-safe design	RAG modules are backend services that can replace mock outputs without rebuilding the existing demo UI.
Multilingual compatibility	English is used as the stable retrieval language for existing stores, while TranslateGemma wraps Arabic input/output.
Legal safety	Legal output is not legal advice, does not determine guilt, does not predict court outcome, and does not guarantee punishment.

3. Inputs from the Classifier and Evidence Packet

The RAG pipeline consumes classifier and telemetry fields rather than raw video as its primary contract. Important fields include verdict, confidence, threshold, fusion gates, graph-quality scores, telemetry

summaries, frame references, packet_summary, narrative, and weapon fields when available. The evidence packet builder deterministically turns telemetry into text; it is not LLM-generated and therefore cannot hallucinate.

Input field	Purpose in RAG
verdict	Controls the explanation and is never changed by any language model.
confidence	Displayed and preserved as classifier confidence.
gate	Explains which streams influenced the classifier: skeleton, interaction, object, visual/ViT.
gqs	Reports evidence quality such as skeleton, interaction, object, person-object and valid-person ratios.
packet_summary	Compact deterministic text used by Explanation RAG, Incident Memory RAG and Legal RAG.
narrative	Grounded language description used for memory and legal context.
weapon_flag / weapon_class	Used as cautious context. Legal RAG uses these but not unreliable exact detected action labels.
frames_ref / frames_vit	Optional visual evidence for VLM narration and future recap display.

4. RAG Store #1: Explanation / Reference RAG

The Explanation RAG grounds the user-facing narrative using the evidence packet and a reference corpus. The corpus includes violence taxonomy, non-violence categories, evaluation caveats, and example explanations. It is intentionally lightweight but still follows the retrieve-then-generate RAG pattern.

- Purpose: produce a 2-4 sentence grounded explanation for a clip classified as violent or non-violent.
- Inputs: verdict, confidence, packet_summary, retrieved reference snippets, and optionally 16 stored frames for a VLM.
- LLM/VLM plan: Qwen2.5-VL-7B-Instruct in 4-bit for frame-aware narration; text-only fallback when VLM is unavailable.
- Guardrails: never change verdict/confidence, do not invent people/objects/weapons, hedge timing/person-level claims as geometry-derived.
- Output: narrative, retrieved references, guardrail_status, and limitations_note.

5. RAG Store #2: Incident Memory / Historical RAG

The Incident Memory RAG is the historical search layer. Every analyzed clip can be saved as a structured incident record and semantically searchable memory. This enables questions such as “tell me about the fight from last week” or “show incidents with a weapon involved”.

Store	Role
SQLite structured metadata	Stores incident_id, timestamp, source, verdict, confidence, weapon_flag, packet_summary, narrative, and filterable metadata.
FAISS/Chroma vector store	Embeds incident narrative and packet summary for semantic retrieval.
Historical search parser	Combines time/verdict/weapon filters with semantic ranking.
Historical summarizer	Answers from stored packet/narrative only and

	includes limitations when details are heuristic.
--	--

The historical RAG avoids re-analyzing the video. It retrieves stored incident records and summarizes only what has already been saved, which improves reproducibility and reduces hallucination risk.

6. RAG Store #3: Legal Consequences RAG

The Legal Consequences RAG is a third independent knowledge store for country-specific legal regulations. It retrieves official or trusted legal texts and generates a cautious summary of possible legal consequences based on selected country, classifier verdict, confidence, packet_summary, narrative, weapon_flag, and weapon_class. It intentionally does not use detected_actions because exact action detection such as punching, kicking, or slapping is not reliable enough for legal grounding.

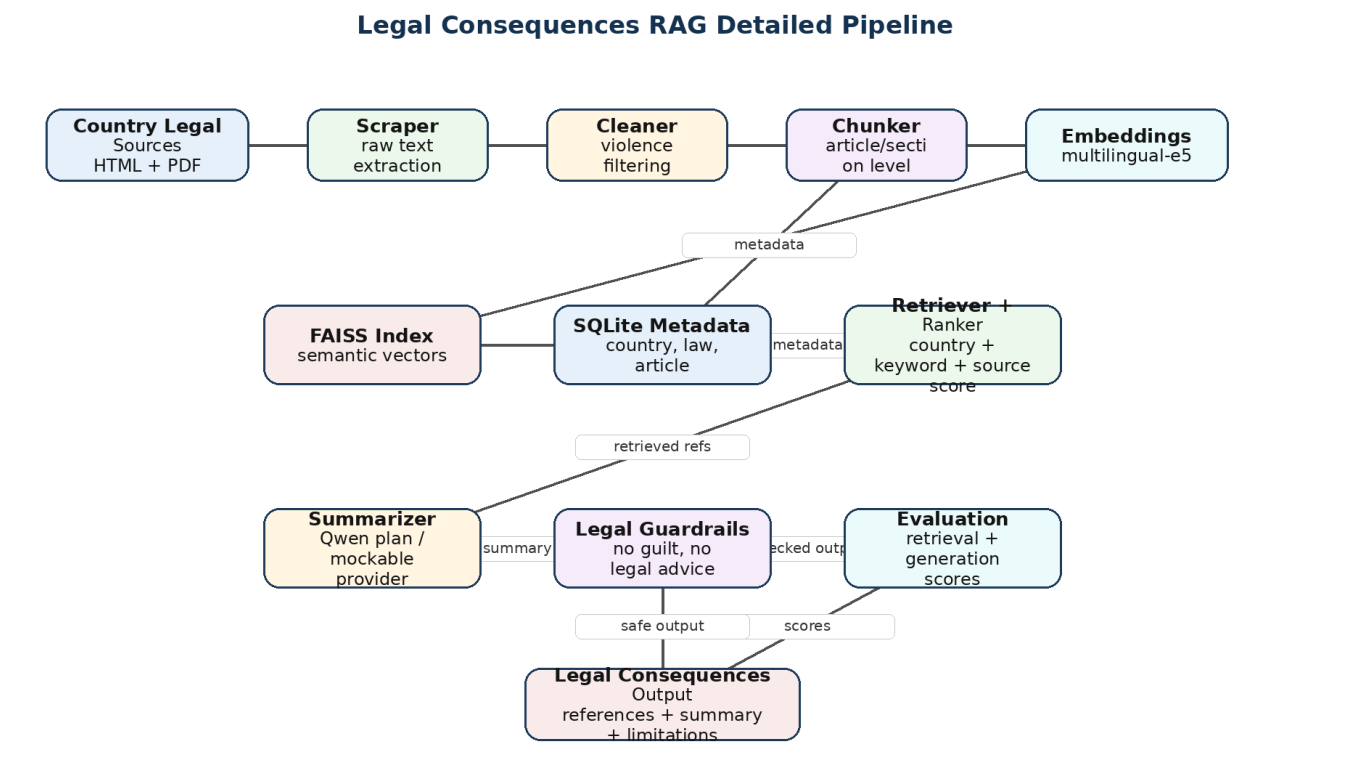


Figure 2. Legal Consequences RAG detailed pipeline.

Legal RAG module	Responsibility
legal_sources.py	Country-specific registry of legal source URLs and metadata.
legal_scraper.py	HTML/PDF raw legal text extraction with safe failure handling.
legal_cleaner.py	Whitespace normalization and violence-related section filtering using deterministic Arabic/English keywords.
legal_chunker.py	Article/section-level chunking while preserving article numbers and metadata.
legal_index.py	FAISS vector index plus SQLite metadata storage.
legal_retrieval.py	Country-filtered retrieval and deterministic ranking/scoring.

legal_summarizer.py	Grounded cautious legal consequence summary using retrieved references only.
legal_guardrails.py	Legal safety checks: no guilt, no legal advice, no guaranteed punishment, no unsupported actions.
legal_evaluation.py	Retrieval and generation scoring helpers.
legal_orchestrator.py	Integration-safe orchestration: retrieval -> summarization -> guardrails -> evaluation.

7. Legal Source Coverage

The initial Legal RAG corpus is designed for multiple jurisdictions: UK, USA/California, Canada, KSA, UAE, and Egypt. Metadata stores country, source_language, law_title, article_number, section_title, violence_category, source_url, official_source, and text. UAE, KSA, Canada, UK, and USA/California are prioritised through official or government legal sources where possible. Egypt uses a temporary source pending replacement with a more official government or official gazette source.

8. Embeddings, Vector Database, and Metadata Storage

Component	Choice	Reason
Reference/Incident embeddings	Sentence-Transformers, bge/e5 family	Small local models are suitable for reference snippets and historical semantic search.
Legal embeddings	intfloat/multilingual-e5-base for testing; intfloat/multilingual-e5-large recommended for production	Supports both Arabic and English legal text.
Vector DB	FAISS CPU	Local, fast, simple, no database server required.
Structured metadata	SQLite	Stores filterable metadata and aligns vector rows with legal/incident records.
Optional vector store	ChromaDB	Can be used if project already standardizes on Chroma collections.

9. LLMs and VLMs Used or Planned

Area	Model	Purpose	Runtime note
Explanation RAG	Qwen2.5-VL-7B-Instruct 4-bit	Narrates from 16 frames, evidence packet, and retrieved reference snippets.	Loaded on demand to protect VRAM.
Text fallback / historical summaries	Qwen2.5-7B-Instruct 4-bit or similar local text LLM	Summarizes stored incident records and supports text-only explanation fallback.	Mocked in tests.
Legal Consequences RAG	Qwen3-8B-Instruct 4-bit planned; fallback Qwen2.5-7B-Instruct 4-bit	Careful legal-text summarization using retrieved chunks only.	Provider interface allows mock summaries in tests.
Arabic/English	google/translategemma-	Translates Arabic	Lazy request-scoped

translation	4b-it	queries to English and user-facing outputs back to Arabic.	loading/unloading.
-------------	-------	--	--------------------

10. Guardrail Design

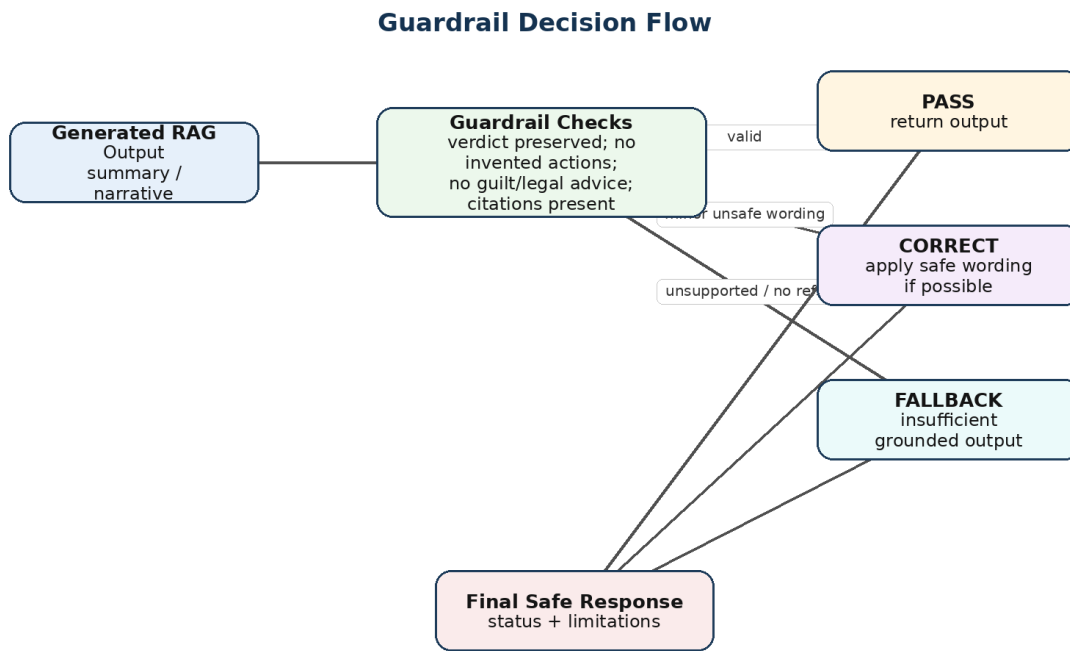


Figure 3. Guardrail decision flow.

- Explanation guardrails prevent changes to verdict and confidence and prevent unsupported people, objects, weapons, timing, responsibility, or exact actions.
- Legal guardrails prevent guilt declarations, legal advice, guaranteed punishment wording, court-outcome predictions, missing references, missing limitations notes, and unsupported exact action claims.
- If output is unsafe but correctable, deterministic correction softens the summary. If not correctable, a fallback response is returned.
- The term `detected_actions` is intentionally excluded from Legal RAG input and output because exact action labels are unreliable.

11. Evaluation Framework

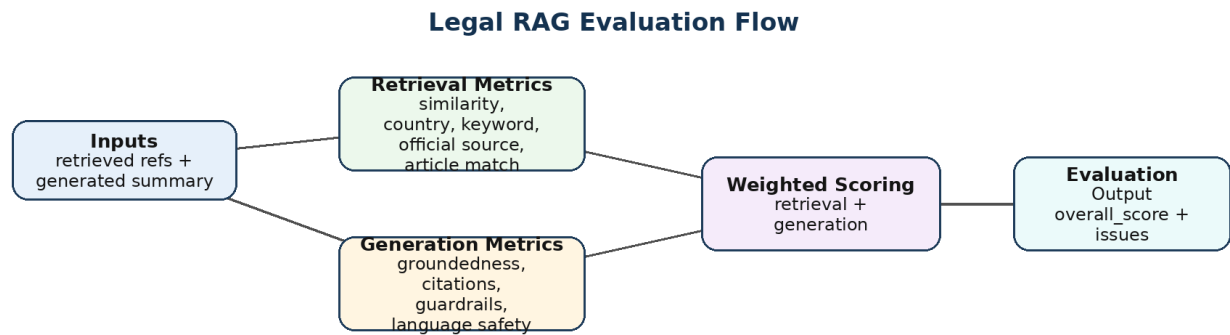


Figure 4. Legal RAG evaluation flow.

Evaluation area	Metrics
Retrieval scoring	top-k similarity, country filter correctness, keyword overlap, source officialness, article-level match.
Generation scoring	groundedness, citation/reference presence, guardrail compliance, language compatibility, unsupported action safety.
Overall score	Combines retrieval_score and generation_score with issue reporting and metric_breakdown.
Mock all-RAG report	Example output includes Legal RAG retrieval score 0.91, generation score 0.89, overall score 0.90, passed=True.

12. Multilingual Support

The existing reference corpus and vector stores remain English. Arabic input is handled by a bilingual wrapper that detects Arabic, translates the query to English, runs the existing RAG, translates user-facing text back to Arabic, and unloads TranslateGemma to free GPU memory. Legal RAG generates English internally and remains compatible with the same final translation wrapper for Arabic output.

13. Mock End-to-End Validation

A mock all-3-RAG runner was added to validate the combined output without real scraping, embeddings, model downloads, vector DB, network, or LLM calls. The mock input simulates a UAE violent incident with a bottle-like object. The report includes Explanation RAG output, Incident Memory RAG output, Legal Consequences RAG output, Legal RAG scores, guardrails, and limitations. This confirms that the backend interfaces are ready for future demo integration.

Test group	Latest result
Legal RAG tests	80 passed across source registry, scraper, cleaner, chunker, index, retrieval, summarizer, guardrails, evaluation, orchestrator, and legal E2E.

All-3-RAG mock E2E	1 passed; script printed combined JSON and regenerated reports/all_rags_mock_report.md.
Offline compatibility	Tests use mocks and do not require network calls, real legal scraping, embedding downloads, or LLM calls.

14. Expected Inputs and Outputs

Legal RAG safe input schema:

```
{
  "country": "UAE",
  "verdict": "violence",
  "confidence": 0.94,
  "packet_summary": "...",
  "narrative": "...",
  "weapon_flag": true,
  "weapon_class": "bottle",
  "language": "en"
}
```

Legal RAG output shape:

```
{
  "legal_consequences": {
    "country": "UAE",
    "query_basis": {
      "verdict": "violence",
      "weapon_flag": true,
      "weapon_class": "bottle"
    },
  },
  "retrieved_legal_references": [
    {
      "law_title": "...",
      "article_number": "...",
      "source_url": "...",
      "snippet": "...",
      "score": 0.87
    }
  ],
  "summary": "According to the retrieved regulation, this type of violent conduct may be subject to...",
  "guardrail_status": "passed",
  "limitations_note": "This is not legal advice and does not determine guilt or predict court outcome."
}
```

15. Tool Stack

Layer	Tools / dependencies
Core implementation	Python 3.10/3.11, Pydantic, NumPy, pandas, python-dotenv.
Text and PDF ingestion	requests, BeautifulSoup4, trafilatura, lxml, PyMuPDF, pypdf.
Embeddings	sentence-transformers, intfloat/multilingual-e5-base or multilingual-e5-large.
Vector search	FAISS CPU; ChromaDB optional.

Structured storage	SQLite, optional SQLAlchemy.
Models	Transformers, Accelerate, BitsAndBytes 4-bit, PyTorch.
Image/VLM support	Pillow for frames; Qwen2.5-VL family for narration.
Testing	pytest with mocks for offline-safe testing.
Version control	Git/GitHub; Codex for focused implementation tasks.

16. Current Limitations

- Classifier output is clip-level only; there is no authoritative per-person or per-frame score unless future model internals are exposed.
- Timing/person-level descriptions are geometry-derived heuristics and must be hedged.
- Legal RAG currently has a source registry and pipeline, but production use requires full corpus ingestion and validation of extracted legal text.
- Egypt legal source should be replaced with a more official government or official gazette source when available.
- The mock all-RAG runner validates integration shape, not real model quality.
- Frontend/demo integration for Legal RAG #3 is intentionally not implemented in the RAG-only phase.

17. Future Work

- Ingest full official legal corpora for all supported countries and schedule periodic re-indexing.
- Connect Legal RAG #3 to the demo backend and add country selector, legal panel, references display, and scores display.
- Add production monitoring for retrieval scores, guardrail failures, and unsupported-action violations.
- Expose model attention or pre-pooling temporal scores to make timing/person explanations more model-grounded.
- Add supervisor-facing report export from actual demo sessions.

18. Source Materials Used

- ARCHITECTURE.md for existing Sara RAG and TranslateGemma architecture notes.
- EXPLANATION_RAG_SYSTEM_readable.pdf for the classifier authority rule, evidence packet, RAG stores #1 and #2, VLM role, and limitations.
- RAG-GUIDELINE.pdf for RAG-only scope, tool stack, team ownership, and demo handoff contract.
- Legal_RAG_Handoff_Instructions.md for Legal RAG modules, schemas, safety notes, and future demo integration call pattern.
- all_rags_mock_report.md for the all-3-RAG mock output and Legal RAG scores.